

UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES
DIRECCIÓN DE POSTGRADO

DIPLOMADO EN DEVOPS & CLOUD NATIVE COMPUTING
MÓDULO 4: CONTINUOUS INTEGRATION & CONTINUOUS DELIVERY (CI/CD)

PROYECTO FINAL — DOCUMENTACIÓN TÉCNICA
MANUAL PASO A PASO: CI/CD CON BLUE-GREEN DEPLOYMENT

Guía Práctica e Integral de Implementación: Branching Strategy, Endpoint /api/instance, GitHub Actions (CI & Release), Nginx Reverse Proxy (Puerto 8085), Scripts de Automatización Bash, Validación E2E y Procedimiento de Rollback

 INTEGRANTES DEL GRUPO:

- Carrion Ojeda Christian Alexander
- Camino Puma Ronald
- Chalar Quiroz David Arturo
- Vaca Diez Soliz Priscila
- Torrez Vaca Andres

Docente / Evaluador: Módulo 4 — DevOps CI/CD

Entorno de Desarrollo: Windows 11 + WSL 2 (Ubuntu Linux)

Proyecto / Repositorio: spring-boot-webapi

Tecnologías Principales: Java 21, Spring Boot 3.2.5, Maven, JUnit 5, JaCoCo, GitHub Actions, Nginx, Bash

Fecha de Presentación: Septiembre 2026

Lugar: Santa Cruz de la Sierra — Bolivia

Manual Paso a Paso — Proyecto Final CI/CD · Módulo 4

Cómo usar este manual

Este manual está diseñado para que **vos escribas cada línea de código y comando**, entiendas qué hace y por qué. **No copies y pegues ciegamente**. Si algo no queda claro, pregunta antes de avanzar.

- ☒ **Checkbox** = paso completado

-  **"¿Por qué?"** = explicación del concepto

-  **"¡Atención!"** = errores comunes a evitar

-  **"Preguntas para reflexionar"** = para que cuestiones lo que estás haciendo

Mapa del proyecto completo

ESTADO ACTUAL del repositorio

- ☒ Spring Boot app básica (WebapiApplication + Calculator)
- ☒ Tests unitarios (CalculatorTest)
- ☒ JaCoCo configurado en pom.xml
- ☒ Workflow CI (maven.yml) - corre en feature/* y main
- ☒ Workflow Release (release.yml) - corre cuando pusheás un tag v*
- ☒ Script de deploy básico (deploy.sh)
- ☒ Branch actual: feature/release

LO QUE FALTA CONSTRUIR

- ☐ Endpoint GET /api/instance para identificar instancias (BLUE/GREEN)
- ☐ Scripts: health-check.sh, traffic-test.sh, rollback.sh
- ☐ Blue-Green Deployment completo con dos instancias
- ☐ Nginx como balanceador de carga
- ☐ Demostración del flujo completo CI → Release → Deploy → Rollback
- ☐ README.md y CHANGELOG.md documentados

ÍNDICE DE ETAPAS

Etapa	Descripción
1	Revisión y comprensión del proyecto actual

2	Estrategia de Branching y Git workflow
3	Agregar endpoint <code>/api/instance</code>
4	Revisar y mejorar el pipeline CI (maven.yml)
5	Gestionar versión v1.1.0 y disparar el Release
6	Instalar y configurar Nginx en WSL
7	Blue-Green Deployment — scripts y configuración
8	Demostrar el flujo completo (deploy → switch → rollback)
9	Pruebas E2E y verificación de tráfico
10	Documentar README.md y CHANGELOG.md

ETAPA 1 — Revisión y comprensión del proyecto actual

Objetivo: Entender qué tenés antes de modificar nada.

1.1 Verificar el entorno de desarrollo

Abrí tu terminal WSL Ubuntu y ejecutá los siguientes comandos **uno por uno**. Observá la salida de cada uno.

```
# ¿Tenés Java instalado?
java -version

# ¿Tenés Maven instalado?
mvn -version

# ¿Tenés Git instalado?
git --version

# ¿Tenés curl instalado?
curl --version
```

 **¿Por qué verificar primero?** Antes de trabajar, siempre confirmás que tu entorno tiene las herramientas necesarias. Si Maven no está instalado y intentás compilar, el error que obtenés puede confundirte porque no sabés si el problema es tu código o el entorno.

💬 Preguntas para reflexionar:

- ¿Qué versión de Java tenés? ¿Coincide con la versión **21** del **pom.xml**?
- ¿Qué pasa si tenés Java 17 pero el proyecto pide Java 21?

1.2 Navegar al proyecto y ver la estructura

```
cd ~/curso-diplomado/uagrm/dev-ops/modulo-4/spring-boot-webapi
ls -la
```

Deberías ver algo así:

```
deploy.sh  docs  mvnw  mvnw.cmd  pom.xml  src  target
```

Ahora explorá la estructura de **src**:

```
find src -type f | sort
```

📖 ¿Qué es cada cosa?

- **src/main/java/** → código fuente de la aplicación
- **src/main/resources/** → configuraciones (como **application.properties**)
- **src/test/java/** → pruebas unitarias
- **pom.xml** → "receta" del proyecto Maven (dependencias, plugins, versión)

1.3 Leer el código fuente existente

Abrí y leé atentamente cada archivo:

```
cat src/main/java/com/cicd/webapi/WebapiApplication.java
```

📖 ¿Qué hace esta clase?

- **@SpringBootApplication** → arranca la aplicación Spring Boot
- **@RestController + @GetMapping("/")** → expone un endpoint HTTP GET en la ruta **/**
- La app tiene 3 endpoints: **/**, **/health**, **/date**

```
cat src/main/java/com/cicd/webapi/Calculator.java
```

📖 **¿Por qué hay una calculadora en una Web API?** Es una clase simple que sirve para demostrar pruebas unitarias con JUnit. La calculadora tiene operaciones matemáticas que son fáciles de testear.

```
cat src/test/java/com/cicd/webapi/CalculatorTest.java
```

Preguntas para reflexionar:

- ¿Qué pasa si cambiás `add(2, 3) == 5` por `add(2, 3) == 6`? ¿Qué esperás que suceda en el pipeline?
- ¿Por qué `testDivide()` verifica que se lanza una excepción cuando el denominador es 0?

1.4 Compilar el proyecto localmente

```
mvn -B clean package -DskipTests --file pom.xml
```

Desglose del comando:

- `mvn` → ejecuta Maven
- `-B` → modo "batch" (sin colores interactivos, ideal para CI)
- `clean` → borra la carpeta `target/` para empezar limpio
- `package` → compila y empaqueta en un `.jar`
- `-DskipTests` → salta las pruebas (solo compilamos para verificar)
- `--file pom.xml` → le dice explícitamente qué `pom.xml` usar

Verificá que se generó el `.jar`:

```
ls target/*.jar
```

Deberías ver: `target/webapi-1.1.0.jar`

 **¡Atención!** Si el build falla, **no avances**. Lee el error completo. Los errores de Maven generalmente dicen exactamente qué está mal.

1.5 Ejecutar los tests localmente

```
mvn -B test --file pom.xml
```

Al finalizar deberías ver algo como:

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Preguntas para reflexionar:

- ¿Cuántos tests corrieron? ¿Por qué 4 y no 3 si solo hay 4 métodos `@Test`?
- ¿Dónde se guardan los reportes de pruebas después de ejecutar `mvn test`?

```
ls target/surefire-reports/
```

1.6 Revisar el estado de Git

```
git status
git branch -a
git log --oneline -8
```

¿Qué es lo que ves?

- `git branch -a` muestra todas las ramas (locales y remotas)
- El historial de commits muestra qué se hizo antes de que arribaras al proyecto
- Actualmente estás en la rama `feature/release`

Preguntas para reflexionar:

- ¿Por qué hay commits en `main` y también en `feature/release`?
- ¿Qué significa el tag `v1.0.0` que ves en el historial?

ETAPA 2 — Estrategia de Branching

Objetivo: Definir y documentar cómo vamos a trabajar con Git branches.

2.1 ¿Qué estrategia vamos a usar?

Para este proyecto usamos una estrategia simplificada basada en **Feature Branching**:

```
main
├── feature/add-instance-endpoint    ← nueva funcionalidad
├── feature/blue-green-scripts      ← scripts de deployment
└── feature/docs-readme            ← documentación
```

Reglas: 1. Nunca trabajar directamente en `main` 2. Cada nueva funcionalidad va en una rama `feature/nombre-descriptivo` 3. Para integrar a `main` → hacer un Pull Request (PR) 4. El pipeline CI corre automáticamente en cualquier `feature/*` y en `main` 5. Los tags (`v1.0.0`, `v1.1.0`) solo se crean desde `main`

 **¿Por qué no trabajamos directamente en `main`?** `main` representa el código "estable". Si vos o un compañero de equipo rompés algo en `main`, nadie puede trabajar hasta que se arregle. Con feature branches, si tu rama falla, solo vos estás bloqueado.

💬 Preguntas para reflexionar:

- ¿Qué pasa si dos personas modifican el mismo archivo en diferentes branches?
- ¿Qué es un "merge conflict" y cuándo ocurre?

2.2 Sincronizar con main antes de empezar

```
# Primero ir a main
git checkout main

# Traer los últimos cambios del remoto
git pull origin main

# Ver que estamos actualizados
git log --oneline -5
```

2.3 Crear la primera rama feature

Ahora vas a crear la rama donde agregaremos el endpoint `/api/instance`:

```
git checkout -b feature/add-instance-endpoint
```

📖 **¿Qué hace `checkout -b`?** Crea una nueva rama a partir de donde estás (`main`) y automáticamente se mueve a esa rama. Es equivalente a:

```
git branch feature/add-instance-endpoint
git checkout feature/add-instance-endpoint
```

Verificá que cambiaste de rama:

```
git branch
```

Deberías ver un asterisco (*) al lado de `feature/add-instance-endpoint`.

ETAPA 3 — Agregar el endpoint `/api/instance`

Objetivo: La aplicación debe poder identificar si es la instancia BLUE (puerto 8081) o GREEN (puerto 8082). Esto es fundamental para demostrar el Blue-Green Deployment.

⚠️ **Nota de tu entorno:** En tu WSL tenés Traefik corriendo en el puerto `80` y contenedores Odoo en `8068–8070`. Por eso usamos puertos libres:

- **Nginx** escucha en → `:8085` (en vez del 80 original)
- **BLUE** corre en → `:8081` (en vez del 8080 original)
- **GREEN** corre en → `:8082` (en vez del 8081 original)

3.1 ¿Por qué necesitamos este endpoint?

En Blue-Green, tenemos **dos instancias idénticas** de la app corriendo en puertos diferentes. Para saber a cuál le llega el tráfico, necesitamos un endpoint que responda con su nombre.

Cuando Nginx reciba una petición en el puerto **8085**, la va a mandar a BLUE o GREEN. Si el endpoint **/api/instance** nos dice "soy GREEN", confirmamos que el routing funciona.

3.2 Modificar `WebapiApplication.java`

Abrí el archivo:

```
nano src/main/java/com/cicd/webapi/WebapiApplication.java
```

O usando VS Code desde Windows: abris el archivo desde el explorador del IDE.

Actualmente el archivo tiene esta estructura:

```
package com.cicd.webapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
public class WebapiApplication {
    public static void main(String[] args) {
        SpringApplication.run(WebapiApplication.class, args);
    }
}

@RestController
class HelloController { ... }

@RestController
class HealthController { ... }

@RestController
class DateController { ... }
```

Tenés que agregar dos cosas: 1. Un nuevo import para **@Value** y **Map** 2. Un nuevo **@RestController** llamado **InstanceController**

El nuevo controller completo que vas a escribir al **final del archivo** (antes del último **}}**):

```
@RestController
class InstanceController {

    @org.springframework.beans.factory.annotation.Value("${app.instance.name:BLUE}")
    private String instanceName;

    @org.springframework.beans.factory.annotation.Value("${server.port:8081}")
    private String serverPort;
```

```
@GetMapping("/api/instance")
public java.util.Map<String, String> instance() {
    return java.util.Map.of(
        "instance", instanceName,
        "port", serverPort
    );
}
```

📝 ¿Por qué `\${server.port:8081}` y no `8080`?

El valor por defecto ahora es **8081** porque ese es el puerto de BLUE en tu entorno. De todos modos, el valor real siempre viene del argumento `--server.port=...` cuando arrancás la app — el default solo aplica si lo corrés sin pasar ningún puerto.

📖 ¿Qué hace `@Value`? Inyecta el valor de una propiedad de configuración. La sintaxis

`${app.instance.name:BLUE}` significa: "leé la propiedad `app.instance.name`, y si no existe, usá **BLUE** como valor por defecto".

Cuando arranquemos la instancia BLUE, le pasaremos `-Dapp.instance.name=BLUE`. Cuando arranquemos GREEN, le pasaremos `-Dapp.instance.name=GREEN`.

💬 Preguntas para reflexionar:

- ¿Por qué `Map.of()` y no un objeto con campos `instance` y `port`?
- ¿Qué devuelve el endpoint en JSON?

3.3 Verificar que compila

```
mvn -B compile --file pom.xml
```

Si ves **BUILD SUCCESS**, el código es válido. Si ves errores, leelos con calma y corregí.

3.4 Escribir un test para el nuevo endpoint

Abrí el archivo de tests de la aplicación:

```
cat src/test/java/com/cicd/webapi/WebapiApplicationTests.java
```

Ahora editá ese archivo para agregar una prueba del endpoint `/api/instance`:

```
nano src/test/java/com/cicd/webapi/WebapiApplicationTests.java
```

Reemplazá el contenido con:

```
package com.cicd.webapi;
```

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
class WebapiApplicationTests {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void contextLoads() {
        // Verifica que el contexto de Spring Boot arranca correctamente
    }

    @Test
    void healthEndpointReturnsOk() throws Exception {
        mockMvc.perform(get("/health"))
            .andExpect(status().isOk())
            .andExpect(content().string("Server Healthy!"));
    }

    @Test
    void instanceEndpointReturnsInstanceInfo() throws Exception {
        mockMvc.perform(get("/api/instance"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.instance").exists())
            .andExpect(jsonPath("$.port").exists());
    }
}
```

 **¿Qué es `MockMvc`?** Es una herramienta de testing de Spring que simula peticiones HTTP sin necesidad de levantar un servidor real. Podés probar tus endpoints sin arrancar la aplicación completa.

 **¿Qué hace `jsonPath("\$.instance").exists()`?** Verifica que en el JSON de respuesta exista una propiedad llamada **instance**. Si el endpoint devuelve `{"instance": "BLUE", "port": "8081"}`, este assertion pasa.

3.5 Ejecutar todos los tests

```
mvn -B test --file pom.xml
```

Deberías ver todos los tests pasando, incluyendo los nuevos.

 **Preguntas para reflexionar:**

- ¿Cuántos tests corren ahora vs. antes? ¿Por qué más?
- ¿Qué pasa si el test falla? ¿Qué significa eso para el pipeline de CI?

3.6 Hacer commit de los cambios

```
# Ver qué archivos cambiaron
git status

# Agregar los archivos al staging area
git add src/main/java/com/cicd/webapi/WebapiApplication.java
git add src/test/java/com/cicd/webapi/WebapiApplicationTests.java

# Hacer el commit con un mensaje descriptivo
git commit -m "feat: add /api/instance endpoint for blue-green identification"
```

 **¿Qué es el "staging area"?** Git tiene tres zonas: tu directorio de trabajo (los archivos que editas), el staging area (lo que marcaste para incluir en el próximo commit) y el repositorio local (los commits guardados). **git add** mueve cambios al staging; **git commit** los guarda permanentemente.

 **Convención de commits ('feat:', 'fix:', 'ci:')** Muchos equipos usan **Conventional Commits**. El prefijo indica el tipo de cambio:

- **feat:** → nueva funcionalidad
- **fix:** → corrección de bug
- **ci:** → cambios en el pipeline/CI
- **docs:** → documentación
- **refactor:** → mejora sin cambiar comportamiento

3.7 Pushear la rama y crear Pull Request

```
git push origin feature/add-instance-endpoint
```

 **¿Qué hace 'push origin'?** Sube tu rama local al repositorio remoto en GitHub. **origin** es el alias que Git usa para referirse al repositorio en GitHub.

Luego, en GitHub: 1. Vas a ver un botón "Compare & pull request" 2. Click en ese botón 3. Ponés un título descriptivo: "feat: add /api/instance endpoint" 4. En la descripción explicás qué hace y por qué 5. Click en "Create Pull Request"

 **¡Atención!** Cuando creás el PR, GitHub Actions va a disparar el pipeline de CI automáticamente sobre tu rama. Esperá a que termine y verificá que todos los checks pasen antes de hacer el merge.

Preguntas para reflexionar:

- ¿Por qué GitHub Actions corre el pipeline en el PR y no solo en main?
- ¿Qué valor tiene que el pipeline corra ANTES del merge?

3.8 Hacer merge del Pull Request

Una vez que el pipeline pasa (☒ verde): 1. Click en "Merge pull request" 2. Click en "Confirm merge" 3. Podés borrar la rama (GitHub te lo sugiere)

```
# Actualizar tu local después del merge
git checkout main
git pull origin main
```

ETAPA 4 — Revisar y entender el pipeline CI

Objetivo: Comprender qué hace el archivo `maven.yml` y por qué está estructurado así.

4.1 Leer el workflow actual

```
cat .github/workflows/maven.yml
```

Desglose del archivo YAML:

```
on:
  push:
    branches:
      - main
      - 'feature/**'
  pull_request:
    branches:
      - main
```

¿Qué significa esto? El pipeline se dispara cuando:

- Alguien hace `push` a `main` o a cualquier rama `feature/algo`
- Alguien abre o actualiza un Pull Request hacia `main`

```
jobs:
  build:
    runs-on: ubuntu-latest
```

¿Qué es `ubuntu-latest`? GitHub Actions corre tu pipeline en una máquina virtual temporaria. Esta máquina no tiene estado — cada run empieza desde cero. Por eso necesitamos el paso de `Setup JDK`.

```

- name: Build with Maven
  run: mvn -B package -DskipTests --file pom.xml

- name: Run tests with Maven
  run: mvn -B test --file pom.xml

- name: Run Code Coverage with Maven
  run: mvn -B verify --file pom.xml

```

💬 Preguntas para reflexionar:

- ¿Por qué se buildea con `-DskipTests` primero, y luego se corren los tests por separado?
- ¿Podría haberse hecho en un solo paso con `mvn clean verify`? ¿Cuáles serían las diferencias?
- ¿Qué hace `mvn verify` que no hace `mvn test`?

📖 **Respuesta:** `mvn verify` ejecuta todas las fases hasta "verify", incluyendo `test` + los goals de plugins como JaCoCo. Es por eso que con `mvn -B verify` se genera el reporte de cobertura. Con solo `mvn test`, JaCoCo no genera el reporte HTML completo.

4.2 Verificar los artefactos publicados

Después de que el pipeline corre en GitHub Actions: 1. Vas a GitHub → tu repositorio 2. Click en la tab "Actions" 3. Click en el último run 4. Al final verás "Artifacts" con los archivos generados:

- `surefire-reports` → reporte de tests JUnit
- `jacoco-report` → reporte HTML de cobertura

Descargá el `jacoco-report` y abrí el `index.html` en tu navegador.

📖 ¿Qué información te da JaCoCo?

- **Classes:** % de clases con al menos un método testeado
- **Methods:** % de métodos ejecutados por los tests
- **Lines:** % de líneas de código ejecutadas
- **Branches:** % de ramas de control de flujo ejecutadas (if/else, switch)

💬 Preguntas para reflexionar:

- ¿Qué cobertura obtenés para `Calculator.java`? ¿Por qué?
- ¿Tenés cobertura del `InstanceController`? ¿Por qué sí o no?
- ¿Una cobertura del 100% garantiza que el código no tiene bugs? ¿Por qué no?

ETAPA 5 — Tagging y GitHub Release

Objetivo: Entender el versionamiento semántico, crear un tag y publicar una Release.

5.1 ¿Qué es Semantic Versioning?

El formato es **MAJOR.MINOR.PATCH**:

Tipo	Cuándo se incrementa
MAJOR	Cambios que rompen compatibilidad (ej: cambiar una API pública)
MINOR	Nueva funcionalidad compatible hacia atrás
PATCH	Corrección de bugs

Ejemplos:

- **v1.0.0** → primera versión estable
- **v1.1.0** → agregamos el endpoint **/api/instance** (nueva funcionalidad)
- **v1.1.1** → corregimos un bug en ese endpoint

5.2 Verificar la versión en pom.xml

El archivo **pom.xml** actualmente tiene configurada la versión **1.1.0**:

```
nano pom.xml
```

Verifiqué que contiene:

```
<groupId>com.cicd</groupId>
<artifactId>webapi</artifactId>
<version>1.1.0</version>
```

 **¿Por qué `1.1.0`?** Porque al haber agregado una nueva funcionalidad (el endpoint **/api/instance** para Blue-Green), siguiendo Semantic Versioning (**MAJOR.MINOR.PATCH**), se incrementa el número **MINOR** (**1.0.0** → **1.1.0**).

5.3 Compilar para verificar que todo sigue funcionando

```
mvn -B clean verify --file pom.xml
```

Verifiqué que el **.jar** tiene el nombre y versión correctos:

```
ls target/*.jar
```

Deberías ver: **target/webapi-1.1.0.jar**

5.4 Commit y push de la versión actualizada

```
git add pom.xml
git commit -m "chore: bump version to 1.1.0 for feature release"
git push origin main
```

5.5 Crear el tag

```
# Crear el tag localmente
git tag v1.1.0

# Pushear el tag a GitHub
git push origin v1.1.0
```

 **¿Qué pasa cuando pusheás un tag `v?`* El archivo `release.yml` tiene configurado:*

```
on:
  push:
    tags:
      - 'v*'
```

Esto significa que cuando se pushea cualquier tag que empiece con `v`, GitHub Actions ejecuta ese workflow automáticamente.

Ese workflow: 1. Hace checkout del código 2. Compila con Maven 3. Obtiene la versión del `pom.xml` (`1.1.0`) 4. Encuentra el `.jar` generado (`webapi-1.1.0.jar`) 5. Crea una GitHub Release con el `.jar` adjunto

5.6 Verificar la Release en GitHub

- 1. Vas a GitHub → tu repositorio
- 2. Click en "Releases" (panel derecho)
- 3. Deberías ver la Release `v1.1.0` con el archivo `webapi-1.1.0.jar` adjunto

 Preguntas para reflexionar:

- ¿Por qué el `.jar` que se publica en la Release viene del pipeline de CI y no de tu máquina local?
- ¿Qué garantía da esto sobre el artifact publicado?
- ¿Qué pasa si modificás el código después de crear el tag? ¿El tag sigue apuntando a lo mismo?

ETAPA 6 — Instalar y configurar Nginx en WSL

Objetivo: Tener Nginx funcionando como balanceador de carga entre las instancias BLUE y GREEN.

6.1 ¿Qué es Nginx y por qué lo usamos?

Nginx es un servidor web que puede funcionar como **proxy inverso** y **balanceador de carga**. En nuestro caso:

NGINX



El usuario siempre va a **localhost:8085**. Nginx decide si mandar la petición a BLUE (puerto 8081) o GREEN (puerto 8082) según la configuración.

⚠ ¿Por qué no usamos el puerto 80?

En tu entorno WSL el puerto **80** ya está ocupado por el contenedor **Traefik** (**0.0.0.0:80->80/tcp**). Si intentamos iniciar Nginx en el 80, fallará con **bind() failed (98: Address already in use)**. Por eso Nginx usará el **puerto 8085**, que está libre.

6.2 Instalar Nginx

```
sudo apt update
sudo apt install nginx -y
```

Verificar que está instalado:

```
nginx -version
```

6.3 Eliminar la configuración default (evita conflicto con puerto 80)

Recién instalado, Nginx viene con un sitio por defecto que intenta escuchar en el **puerto 80**. Como en tu WSL el puerto 80 ya está ocupado por **Traefik**, si intentás iniciarlo sin quitar esto, fallará con **bind() failed (98: Address already in use)**.

Por eso, lo primero que hacemos es eliminar ese archivo:

```
sudo rm -f /etc/nginx/sites-enabled/default
```

6.4 Configurar Nginx para Blue-Green en el puerto 8085

Nginx guarda sus configuraciones en **/etc/nginx/sites-available/**. Vamos a crear una configuración para nuestro proyecto:

```
sudo nano /etc/nginx/sites-available/spring-boot-webapi
```

Escribí esta configuración:

```
upstream blue_green {
    server 127.0.0.1:8081; # instancia BLUE (activa por defecto)
}

server {
    listen 8085;          # ← puerto 8085 (libre en tu WSL)
```

```
server_name localhost;

location / {
    proxy_pass http://blue_green;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Cambios respecto al manual original:

- **listen 8085** en vez de **listen 80** → el 80 lo ocupa Traefik en tu entorno
- **server 127.0.0.1:8081** en vez de **8080** → BLUE ahora corre en 8081

 **¿Qué es `upstream`?** Define un grupo de servidores a los que Nginx puede redirigir el tráfico. El nombre **blue_green** es un alias que usamos en **proxy_pass**.

 **¿Qué hace `proxy\_pass`?** Le dice a Nginx: "cuando llegue una petición, reenvíala a este grupo de servidores". El cliente (tu navegador) cree que habla con Nginx, pero en realidad Nginx habla con tu app Spring Boot.

6.5 Activar y verificar la configuración

```
# Crear un enlace simbólico en sites-enabled
sudo ln -sf /etc/nginx/sites-available/spring-boot-webapi /etc/nginx/sites-enabled/

# Verificar que la configuración es válida (¡importante!)
sudo nginx -t
```

Deberías ver:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

 **¿Qué hace `nginx -t`?** Verifica la sintaxis del archivo de configuración antes de arrancar. Si hay un error, Nginx te dice en qué línea.

6.6 Iniciar o Reiniciar Nginx

Ahora que Nginx ya está configurado en el puerto libre **8085** y sin conflicto con el **80**:

```
# Iniciar (o reiniciar si ya estaba en ejecución):
sudo service nginx restart
```

Verificar que corre correctamente:

```
sudo service nginx status
```

Deberías ver **Active: active (running)**.

6.7 Reactivar Traefik (si lo habías detenido)

Si tuviste que detener temporalmente tu contenedor Traefik para hacer pruebas, podés volver a encenderlo con tranquilidad, ya que Nginx ahora escucha en el puerto **8085** y no tocará el **80**:

```
docker start traefik
```

6.8 Probar el acceso a Nginx

Probá la conexión desde tu terminal WSL:

```
curl http://localhost:8085
```

O desde tu navegador en Windows: **http://localhost:8085**

¿Qué respuesta debes esperar?

- **`502 Bad Gateway`** → **¡ES CORRECTO!** Significa que Nginx está activo y funcionando en el puerto 8085, pero como todavía no arrancamos la app Spring Boot en el puerto 8081, Nginx no tiene a quién pasarle la petición.
- **`ERR\_CONNECTION\_REFUSED`** → **Incorrecto.** Significa que Nginx no está corriendo en el puerto 8085 (revisar **sudo service nginx status** y **sudo nginx -t**).

¿Por qué `sudo`? Nginx necesita privilegios para arrancar como daemon y escribir en **/run/nginx.pid**.

 **¡Atención en WSL!** En WSL, los servicios no se inician automáticamente cuando abrís la terminal. Cada vez que cerrés y abrás WSL, tenés que hacer **sudo service nginx start** manualmente (o configurar systemd si tu versión de WSL lo soporta).

ETAPA 7 — Scripts de Blue-Green Deployment

Objetivo: Crear los scripts de automatización del deployment.

7.1 Estructura de directorios para el deployment

Primero, crear la estructura donde vivirán las instancias:

```
mkdir -p ~/blue-green/blue/logs
mkdir -p ~/blue-green/green/logs
mkdir -p ~/blue-green/releases
```

📖 ¿Por qué esta estructura?

- `~/blue-green/blue/` → directorio de la instancia BLUE
- `~/blue-green/green/` → directorio de la instancia GREEN
- `~/blue-green/releases/` → versiones del `.jar` descargadas de las Releases

7.2 Crear la estructura de scripts en el repositorio

```
mkdir -p scripts
```

Script 1: `scripts/deploy-blue-green.sh`

```
nano scripts/deploy-blue-green.sh
```

Escribí el siguiente script **línea por línea**, entendiendo cada parte:

```
#!/usr/bin/env bash
# =====
# deploy-blue-green.sh – Blue-Green Deployment Script
# Uso: ./scripts/deploy-blue-green.sh <path-al-jar> <blue|green>
# =====
set -euo pipefail

# — Argumentos —————
JAR_PATH="${1:-}"
TARGET_ENV="${2:-green}" # La instancia que vamos a actualizar

if [[ -z "$JAR_PATH" ]]; then
    echo "❌ Error: Debes especificar el path al JAR"
    echo "  Uso: $0 <path-al-jar> <blue|green>"
    exit 1
fi

# — Configuración —————
BASE_DIR="$HOME/blue-green"
BLUE_DIR="$BASE_DIR/blue"
GREEN_DIR="$BASE_DIR/green"
JAR_NAME="app.jar"

# Puerto según la instancia
# ⚠️ Puertos ajustados a tu entorno (Traefik ocupa el 80, Odoo ocupa 8068-8070)
if [[ "$TARGET_ENV" == "blue" ]]; then
    DEPLOY_DIR="$BLUE_DIR"
    PORT=8081
else
    DEPLOY_DIR="$GREEN_DIR"
    PORT=8082
fi

echo "=====
echo "🚀 Blue-Green Deployment"
echo "  Instancia destino: ${TARGET_ENV^^}"
echo "  Puerto: $PORT"
echo "  JAR: $JAR_PATH"
```

```

echo "===== "

# — Detener instancia anterior —————
echo ""
echo "🔍 Verificando si hay una instancia corriendo en puerto $PORT..."
PID=$(lsof -ti tcp:$PORT 2>/dev/null || true)

if [[ -n "$PID" ]]; then
    echo "    PID encontrado: $PID – deteniendo..."
    kill "$PID"
    sleep 3
    echo "    ✅ Instancia detenida"
else
    echo "    ⓘ No hay ninguna instancia corriendo en el puerto $PORT"
fi

# — Copiar el nuevo JAR —————
echo ""
echo "📦 Copiando nuevo JAR..."
cp "$JAR_PATH" "$DEPLOY_DIR/$JAR_NAME"
chmod 755 "$DEPLOY_DIR/$JAR_NAME"
echo "    ✅ JAR copiado en $DEPLOY_DIR/$JAR_NAME"

# — Arrancar la nueva instancia —————
echo ""
echo "▶ Arrancando instancia ${TARGET_ENV^^} en puerto $PORT..."
nohup java -jar "$DEPLOY_DIR/$JAR_NAME" \
    --server.port="$PORT" \
    --app.instance.name="${TARGET_ENV^^}" \
    > "$DEPLOY_DIR/logs/app.log" 2>&1 &

JAVA_PID=$!
echo "    PID del proceso Java: $JAVA_PID"

# — Health Check —————
echo ""
echo "🕒 Esperando que la instancia esté lista..."
MAX_RETRIES=20
for i in $(seq 1 $MAX_RETRIES); do
    if curl -sf "http://localhost:$PORT/health" > /dev/null 2>&1; then
        echo "    ✅ Instancia ${TARGET_ENV^^} lista (intento $i/$MAX_RETRIES)"
        break
    fi
    echo "    ⌚ Intento $i/$MAX_RETRIES – esperando 3 segundos..."
    sleep 3

    if [[ $i -eq $MAX_RETRIES ]]; then
        echo "    ❌ La instancia no respondió después de $MAX_RETRIES intentos"
        echo "    📖 Últimas líneas del log:"
        tail -20 "$DEPLOY_DIR/logs/app.log"
        exit 1
    fi
done

echo ""
echo "🎉 Deployment de instancia ${TARGET_ENV^^} completado exitosamente"
echo "    Verificá con: curl http://localhost:$PORT/api/instance"

```

📖 ¿Qué es `set -euo pipefail`?

- **-e** → el script termina si cualquier comando falla (no continúa ignorando errores)
- **-u** → si usás una variable no definida, termina con error en vez de usar string vacío
- **-o pipefail** → si un comando en un pipe falla, el pipe completo falla

Son buenas prácticas de seguridad en scripts Bash.

Script 2: `scripts/health-check.sh`

```
nano scripts/health-check.sh
```

```
#!/usr/bin/env bash
# =====
# health-check.sh – Verifica el estado de las instancias BLUE y GREEN
# Uso: ./scripts/health-check.sh
# =====
set -euo pipefail

BLUE_PORT=8081 # ← ajustado (8080 puede conflictuar con Traefik)
GREEN_PORT=8082 # ← ajustado

check_instance() {
    local name="$1"
    local port="$2"

    echo "_____ "
    echo "🔍 Verificando instancia $name (puerto $port)..."

    # Health check
    HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
        "http://localhost:$port/health" 2>/dev/null || echo "000")

    if [[ "$HTTP_STATUS" == "200" ]]; then
        echo "✅ Estado: HEALTHY (HTTP $HTTP_STATUS)"

        # Obtener info de la instancia
        INSTANCE_INFO=$(curl -s "http://localhost:$port/api/instance" 2>/dev/null || echo '{}')
        echo "📄 Info: $INSTANCE_INFO"
    else
        echo "❌ Estado: DOWN (HTTP $HTTP_STATUS)"
    fi
}

echo "===== "
echo "📁 Health Check – Blue-Green Instances"
echo "===== "

check_instance "BLUE" "$BLUE_PORT"
check_instance "GREEN" "$GREEN_PORT"

echo "_____ "
echo ""
echo "💡 Para ver el tráfico actual de Nginx:"
echo "curl http://localhost:8085/api/instance"
```

Script 3: `scripts/traffic-test.sh`

```
nano scripts/traffic-test.sh
```

```
#!/usr/bin/env bash
# =====
# traffic-test.sh – Envía múltiples requests para observar el balanceo
# Uso: ./scripts/traffic-test.sh [numero-de-requests]
# =====

REQUESTS="${1:-20}"
NGINX_URL="http://localhost:8085/api/instance" # ← Nginx corre en 8085, no en 80

echo "=====
echo "📡 Traffic Test – $REQUESTS requests a Nginx"
echo "  URL: $NGINX_URL (Nginx en puerto 8085)"
echo "=====
echo ""

BLUE_COUNT=0
GREEN_COUNT=0

for i in $(seq 1 "$REQUESTS"); do
  RESPONSE=$(curl -s "$NGINX_URL" 2>/dev/null || echo '{"instance":"ERROR","port": "?"}')
  INSTANCE=$(echo "$RESPONSE" | grep -o '"instance": "[^"]*"' | cut -d'"' -f4)
  PORT=$(echo "$RESPONSE" | grep -o '"port": "[^"]*"' | cut -d'"' -f4)

  printf "  Request #%02d → %-6s (puerto %s)\n" "$i" "$INSTANCE" "$PORT"

  if [[ "$INSTANCE" == "BLUE" ]]; then
    BLUE_COUNT=$((BLUE_COUNT + 1))
  elif [[ "$INSTANCE" == "GREEN" ]]; then
    GREEN_COUNT=$((GREEN_COUNT + 1))
  fi

  sleep 0.2
done

echo ""
echo "=====
echo "📊 Resultados:"
echo "  BLUE recibió: $BLUE_COUNT requests ($( BLUE_COUNT * 100 / REQUESTS )%)"
echo "  GREEN recibió: $GREEN_COUNT requests ($( GREEN_COUNT * 100 / REQUESTS )%)"
echo "=====
```

Script 4: `scripts/switch-traffic.sh`

```
nano scripts/switch-traffic.sh
```

```
#!/usr/bin/env bash
# =====
# switch-traffic.sh – Cambia el tráfico de Nginx hacia una instancia
# Uso: ./scripts/switch-traffic.sh <blue|green>
# =====
```

```

set -euo pipefail

TARGET="${1:-}"

if [[ -z "$TARGET" || ( "$TARGET" != "blue" && "$TARGET" != "green" ) ]]; then
    echo "❌ Error: Debes especificar 'blue' o 'green'"
    echo "    Uso: $0 <blue|green>"
    exit 1
fi

# ⚠️ Puertos ajustados a tu entorno
if [[ "$TARGET" == "blue" ]]; then
    PORT=8081
else
    PORT=8082
fi

NGINX_CONF="/etc/nginx/sites-available/spring-boot-webapi"

echo "====="
echo "🔄 Cambiando tráfico hacia: ${TARGET^^} (puerto $PORT)"
echo "====="

# Reemplazar el upstream en la configuración de Nginx
sudo sed -i "s|server 127.0.0.1:[0-9]*;|server 127.0.0.1:$PORT;|g" "$NGINX_CONF"

# Verificar que la configuración es válida
sudo nginx -t

# Recargar Nginx (sin downtime)
sudo nginx -s reload

echo ""
echo "✅ Tráfico redirigido hacia ${TARGET^^}"
echo "    Verificá con: curl http://localhost:8085/api/instance"

```

Script 5: `scripts/rollback.sh`

```
nano scripts/rollback.sh
```

```

#!/usr/bin/env bash
# =====
# rollback.sh – Rollback hacia la instancia estable anterior
# Uso: ./scripts/rollback.sh <blue|green>
#       (especificás la instancia ESTABLE, no la que falló)
# =====
set -euo pipefail

STABLE_ENV="${1:-blue}"

echo "====="
echo "🔄 ROLLBACK – Revertiendo a instancia ${STABLE_ENV^^}"
echo "====="
echo ""

# Cambiar el tráfico hacia la instancia estable
bash "$(dirname "$0")/switch-traffic.sh" "$STABLE_ENV"

```

```

echo ""
echo "🔍 Verificando que la instancia estable responde..."
# ⚠️ Puertos ajustados a tu entorno
if [[ "$STABLE_ENV" == "blue" ]]; then
    STABLE_PORT=8081
else
    STABLE_PORT=8082
fi

for i in {1..5}; do
    RESPONSE=$(curl -s "http://localhost:8085/api/instance" 2>/dev/null || echo '{}')
    echo "    Request $i → $RESPONSE"
    sleep 1
done

echo ""
echo "✅ Rollback completado. Tráfico apuntando a ${STABLE_ENV^^}"
echo ""
echo "⚠️ ACCIÓN REQUERIDA: Investigar y corregir la instancia que falló antes del próximo
deploy."

```

7.3 Dar permisos de ejecución a los scripts

```

chmod +x scripts/deploy-blue-green.sh
chmod +x scripts/health-check.sh
chmod +x scripts/traffic-test.sh
chmod +x scripts/switch-traffic.sh
chmod +x scripts/rollback.sh

```

💬 Preguntas para reflexionar:

- ¿Por qué necesitamos **chmod +x**? ¿Qué significa el **+x**?
- ¿Qué pasaría si intentás ejecutar un script sin el permiso de ejecución?

7.4 Commit de los scripts

```

git add scripts/
git commit -m "feat: add blue-green deployment scripts"
git push origin main

```

ETAPA 8 — Demostrar el flujo Blue-Green completo

Objetivo: Ejecutar el flujo completo: arrancar BLUE, deployar GREEN, cambiar tráfico, verificar, y hacer rollback.

8.1 Descargar el JAR de la Release

Primero necesitamos el JAR publicado en GitHub:

```
# Crear directorio para la release
mkdir -p ~/blue-green/releases

# Descargar el JAR de la Release v1.1.0 (que está publicada en tu GitHub)
curl -L \
  "https://github.com/R0nald12345/spring-boot-webapi-practica/releases/download/v1.1.0/webapi-1.1.0.jar" \
  -o ~/blue-green/releases/webapi-1.1.0.jar
```

 **¿Por qué descargamos de la Release y no compilamos localmente?** El principio de CI/CD dice: "el artifact que se testea es el mismo que se despliega". Si compilaras localmente, no tenés garantía de que sea exactamente el mismo binary que pasó por el pipeline de CI.

Verificar que se descargó (debe pesar alrededor de 18 MB):

```
ls -lh ~/blue-green/releases/
```

8.2 Arrancar la instancia BLUE

```
bash scripts/deploy-blue-green.sh ~/blue-green/releases/webapi-1.1.0.jar blue
```

Verificar que BLUE responde:

```
curl http://localhost:8081/api/instance
```

Debería devolver: `{"instance": "BLUE", "port": "8081"}`

8.3 Configurar Nginx para apuntar a BLUE

```
bash scripts/switch-traffic.sh blue
```

Verificar a través de Nginx (puerto 8085):

```
curl http://localhost:8085/api/instance
```

Debería devolver: `{"instance": "BLUE", "port": "8081"}`

8.4 Simular un nuevo deploy en GREEN

Imaginemos que hicimos un cambio, el pipeline generó **v1.1.0**. Vamos a deployar en GREEN:

```
bash scripts/deploy-blue-green.sh ~/blue-green/releases/webapi-1.1.0.jar green
```

Verificar que GREEN responde directamente:

```
curl http://localhost:8082/api/instance
```

Debería devolver: `{"instance": "GREEN", "port": "8082"}`

En este momento, BLUE sigue activo y recibiendo tráfico, mientras GREEN está lista pero sin tráfico.

8.5 Verificar ambas instancias con health-check

```
bash scripts/health-check.sh
```

Deberías ver que ambas instancias están **HEALTHY**.

8.6 Prueba del tráfico antes del switch

```
bash scripts/traffic-test.sh 10
```

Deberías ver que **todos los requests van a BLUE** (porque Nginx apunta a :8081).

8.7 Cambiar el tráfico a GREEN

```
bash scripts/switch-traffic.sh green
```

8.8 Verificar el tráfico después del switch

```
bash scripts/traffic-test.sh 10
```

Ahora **todos los requests deben ir a GREEN**. ¡Este es el Blue-Green Switch!

Preguntas para reflexionar:

- ¿Hubo algún momento de downtime durante el switch?
- ¿Qué pasaría si GREEN hubiera fallado el health check?
- ¿Por qué sigue corriendo BLUE aunque no recibe tráfico?

8.9 Simular un fallo y hacer Rollback

Simulemos que GREEN tiene un problema. Paramos GREEN manualmente:

```
# Encontrar y matar el proceso en puerto 8082 (GREEN en tu entorno)
kill $(lsof -ti tcp:8082)
```

Verificar que GREEN ya no responde:

```
curl http://localhost:8082/health
# Debería dar un error de conexión
```

Verificar el estado de ambas instancias:

```
bash scripts/health-check.sh
```

Hacer rollback hacia BLUE:

```
bash scripts/rollback.sh blue
```

Verificar que el tráfico volvió a BLUE:

```
bash scripts/traffic-test.sh 10
```

Preguntas para reflexionar:

- ¿Cuánto tiempo pasó desde que GREEN falló hasta que el tráfico volvió a BLUE?
- ¿Quién detecta el fallo en este esquema? ¿Es automático o manual?
- ¿Cómo automatizarías la detección del fallo y el rollback?

ETAPA 9 — Pruebas E2E y verificación del servicio

Objetivo: Crear una prueba end-to-end básica que valide el comportamiento real de la aplicación.

9.1 ¿Qué es una prueba E2E?

Las pruebas **End-to-End** (de extremo a extremo) verifican el flujo completo de la aplicación desde el punto de vista del usuario. A diferencia de las pruebas unitarias (que testean clases aisladas), las E2E testean que el sistema completo funciona.

En nuestro caso, una prueba E2E verifica que: 1. La app responde en el puerto correcto 2. Los endpoints devuelven las respuestas esperadas 3. Nginx balancea el tráfico correctamente

9.2 Crear el script de E2E

```
nano scripts/e2e-test.sh
```

```
#!/usr/bin/env bash
# =====
# e2e-test.sh – Pruebas End-to-End básicas
# Uso: ./scripts/e2e-test.sh <blue|green|nginx>
# =====
set -euo pipefail

TARGET="${1:-nginx}"

# Determinar URL base
case "$TARGET" in
    blue) BASE_URL="http://localhost:8081" ;; # BLUE en tu entorno
    green) BASE_URL="http://localhost:8082" ;; # GREEN en tu entorno
    nginx) BASE_URL="http://localhost:8085" ;; # Nginx en tu entorno (no usa el 80)
    *)
        echo "❌ Uso: $0 <blue|green|nginx>"
        exit 1
        ;;
esac
```

```

esac

PASS=0
FAIL=0

assert_equals() {
    local test_name="$1"
    local expected="$2"
    local actual="$3"

    if [[ "$actual" == "$expected" ]]; then
        echo "    ☑ PASS: $test_name"
        PASS=$((PASS + 1))
    else
        echo "    ✗ FAIL: $test_name"
        echo "        Esperado: '$expected'"
        echo "        Obtenido: '$actual'"
        FAIL=$((FAIL + 1))
    fi
}

assert_contains() {
    local test_name="$1"
    local expected="$2"
    local actual="$3"

    if echo "$actual" | grep -q "$expected"; then
        echo "    ☑ PASS: $test_name"
        PASS=$((PASS + 1))
    else
        echo "    ✗ FAIL: $test_name"
        echo "        Se esperaba que contenga: '$expected'"
        echo "        Obtenido: '$actual'"
        FAIL=$((FAIL + 1))
    fi
}

echo "=====
echo "🔧 E2E Tests – Objetivo: $TARGET"
echo "  URL Base: $BASE_URL"
echo "=====
echo ""

# Test 1: Root endpoint
echo "Test 1: Root endpoint"
RESPONSE=$(curl -s "$BASE_URL/" 2>/dev/null || echo "FAILED")
assert_equals "GET / devuelve 'Hello CI/CD World!'" "Hello CI/CD World!" "$RESPONSE"

# Test 2: Health endpoint
echo ""
echo "Test 2: Health endpoint"
RESPONSE=$(curl -s "$BASE_URL/health" 2>/dev/null || echo "FAILED")
assert_equals "GET /health devuelve 'Server Healthy!'" "Server Healthy!" "$RESPONSE"

# Test 3: Instance endpoint exists
echo ""
echo "Test 3: Instance endpoint"
RESPONSE=$(curl -s "$BASE_URL/api/instance" 2>/dev/null || echo "FAILED")
assert_contains "GET /api/instance contiene 'instance'" "instance" "$RESPONSE"
assert_contains "GET /api/instance contiene 'port'" "port" "$RESPONSE"

```

```
# Test 4: HTTP status codes
echo ""
echo "Test 4: HTTP Status Codes"
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BASE_URL/health")
assert_equals "GET /health retorna HTTP 200" "200" "$STATUS"

STATUS_404=$(curl -s -o /dev/null -w "%{http_code}" "$BASE_URL/ruta-inexistente")
assert_equals "GET /ruta-inexistente retorna HTTP 404" "404" "$STATUS_404"

# Resumen
echo ""
echo "=====
echo "📊 Resultados E2E:"
echo "    ✅ Pasaron: $PASS"
echo "    ❌ Fallaron: $FAIL"
echo "=====

if [[ $FAIL -gt 0 ]]; then
    echo ""
    echo "❌ E2E Tests FALLARON – No proceder con el switch de tráfico"
    exit 1
else
    echo ""
    echo "✅ Todos los E2E Tests pasaron – Instancia apta para recibir tráfico"
    exit 0
fi
```

```
chmod +x scripts/e2e-test.sh
```

9.3 Ejecutar los tests E2E

```
# Testear instancia BLUE directamente
bash scripts/e2e-test.sh blue

# Testear instancia GREEN directamente
bash scripts/e2e-test.sh green

# Testear a través de Nginx
bash scripts/e2e-test.sh nginx
```

📖 **¿Por qué testeamos antes del switch?** El flujo correcto es: 1. Deployar en GREEN 2. Correr E2E tests en GREEN directamente (puerto **8082** en tu entorno) 3. Si pasan → hacer switch de tráfico 4. Si fallan → rollback, no hacemos switch

Esto garantiza que nunca enviamos tráfico a una instancia rota.

9.4 Commit de los E2E tests

```
git add scripts/e2e-test.sh
git commit -m "test: add e2e test script for blue-green validation"
git push origin main
```

ETAPA 10 — Documentar README.md y CHANGELOG.md

Objetivo: Crear la documentación requerida por el proyecto.

10.1 Crear `README.md`

```
nano README.md
```

Escribí el README con estas secciones mínimas requeridas:

```
# Spring Boot WebAPI – CI/CD con Blue-Green Deployment

## Descripción
...

## Arquitectura
...

## Tecnologías
- Java 21, Spring Boot 3.2.5, Maven
- JUnit 5, JaCoCo
- GitHub Actions (CI/CD)
- Nginx (balanceador de carga)
- Bash scripts
- WSL Ubuntu (infraestructura local)

## Estrategia de Branching
- `main` → rama estable
- `feature/*` → desarrollo de funcionalidades

## Estrategia de Tagging (Semantic Versioning)
- `MAJOR.MINOR.PATCH`
- Tags se crean desde main después de merge

## Pipeline CI/CD
...

## Deployment – Blue-Green
...

## Scripts
| Script | Descripción |
|-----|-----|
| `scripts/deploy-blue-green.sh <jar> <blue\|green>` | Deploya en la instancia indicada |
| `scripts/health-check.sh` | Verifica el estado de ambas instancias |
| `scripts/traffic-test.sh [N]` | Envía N requests y muestra distribución |
| `scripts/switch-traffic.sh <blue\|green>` | Cambia tráfico en Nginx |
| `scripts/rollback.sh <blue\|green>` | Rollback a la instancia estable |
| `scripts/e2e-test.sh <blue\|green\|nginx>` | Pruebas E2E |

## Rollback
...
```

10.2 Crear `CHANGELOG.md`

```
nano CHANGELOG.md
```

```
# Changelog

## [1.1.0] - 2026-09-XX
### Added
- Endpoint `/api/instance` para identificación en Blue-Green
- Scripts de Blue-Green Deployment
- Script de Health Check
- Script de Traffic Test
- Script de E2E Tests
- Script de Rollback
- Configuración Nginx para balanceo de carga

## [1.0.0] - 2026-09-XX
### Added
- Aplicación Spring Boot base
- Endpoints: `/`, `/health`, `/date`
- Calculator con operaciones básicas
- Tests unitarios con JUnit 5
- Code Coverage con JaCoCo
- Pipeline CI con GitHub Actions
- GitHub Release con JAR adjunto
```

10.3 Commit final

```
git add README.md CHANGELOG.md
git commit -m "docs: add README and CHANGELOG"
git push origin main
```

FLUJO COMPLETO DE DEMOSTRACIÓN

Para la entrega final, debes poder demostrar este flujo de principio a fin:

1. `git checkout -b feature/mi-cambio`
2. [hacer cambios en el código]
3. `git push` → CI pipeline corre automáticamente
4. Pull Request → CI vuelve a correr
5. Merge a main
6. `git tag v1.1.0 && git push origin v1.1.0`
7. Release pipeline genera el JAR y lo publica
8. Descargar JAR de la Release
9. `bash scripts/deploy-blue-green.sh webapi-1.1.0.jar green`
10. `bash scripts/health-check.sh`
11. `bash scripts/e2e-test.sh green`
12. `bash scripts/switch-traffic.sh green`
13. `bash scripts/traffic-test.sh 20`
14. [si hay problema] `bash scripts/rollback.sh blue`

CHECKLIST FINAL DEL PROYECTO

Código

- [] Endpoint `/api/instance` funcionando
- [] Tests unitarios pasando
- [] JaCoCo generando reporte de cobertura

GitHub

- [] Branch `main` protegido con reglas
- [] Al menos un PR completo en el historial
- [] Tags siguiendo Semantic Versioning
- [] Al menos una GitHub Release con `.jar` publicado

Pipeline CI

- [] `maven.yml` corre en `feature/*` y en PRs a `main`
- [] Build, test, coverage y upload de artifacts
- [] `release.yml` corre al pushear un tag `v*`

Infraestructura local

- [] Nginx instalado y configurado (escuchando en `:8085`)
- [] Instancia BLUE corriendo en `:8081`
- [] Instancia GREEN corriendo en `:8082`
- [] Nginx balanceando hacia la instancia activa

Scripts

- [] `deploy-blue-green.sh` funciona
- [] `health-check.sh` funciona
- [] `traffic-test.sh` funciona
- [] `switch-traffic.sh` funciona
- [] `rollback.sh` funciona
- [] `e2e-test.sh` funciona

Documentación

- [] `README.md` completo
- [] `CHANGELOG.md` con historial de versiones

PREGUNTAS FRECUENTES

¿Qué hago si el JAR no arranca?

```
# Ver el log completo
tail -100 ~/blue-green/blue/logs/app.log
# o
tail -100 ~/blue-green/green/logs/app.log
```

¿Cómo sé qué puerto está usando cada proceso Java?

```
lsof -i :8081    # BLUE en tu entorno  
lsof -i :8082    # GREEN en tu entorno
```

¿Cómo detengo una instancia manualmente?

```
kill $(lsof -ti tcp:8081)  # detener BLUE  
kill $(lsof -ti tcp:8082)  # detener GREEN
```

¿Por qué Nginx dice "502 Bad Gateway"? Nginx puede alcanzar el puerto pero la app Java no está respondiendo. Verificá con [health-check.sh](#).

¿Cómo verifico que Nginx está corriendo en el puerto correcto?

```
ss -tulpn | grep 8085    # debe aparecer el proceso nginx  
curl http://localhost:8085/health
```

¿Cómo recargo Nginx sin downtime?

```
sudo nginx -s reload    # ← recarga graceful, sin downtime  
# NO usar: sudo service nginx restart (eso tiene un momento de downtime)
```